Pract-1A Encryption/Decryption

```
def encrypt(text,s):
    result = ""
    for i in range(len(text)):
        char = text[i]
        if (char.isupper()):
            result += chr((ord(char) + s - 65) % 26 + 65)
        else:
            result += chr((ord(char) + s - 97) % 26 +97)
    return result
def decrypt(text,s):
    result = ""
    for i in range(len(text)):
        char = text[i]
        if (char.isupper()):
            result += chr((ord(char) + s - 65) % 26 + 65)
        else:
            result += chr((ord(char) + s - 97) % 26 +97)
    return result
while True:
    print("\n1.Encryption")
    print("2.Decryption")
    print("3.Exit")
    choice = int(input("Enter the choice: "))
    if choice == 1:
        text = input("Enter the text to encrypt: ")
        s = 3
        print("Text: " + text)
        str(s)
        print("Cipher: " + encrypt(text,s))
```

```
    elif choice == 2:
        text = input("Enter the text to Decrypt: ")
        s = -3
        print("Text: " + text)
        str(s)
        print("Cipher: " + decrypt(text,s))
    elif choice == 3:
        print("Exit")
        break
```

---

pract-1B Rail-Fence Cipher

```
s=input("Enter a string: ")
k=int(input("Enter a Key: "))
enc=[[" " for i in range(len(s))]for j in range(k)]
row = 0
flag = 0
for i in range(len(s)):
    enc[row][i]=s[i]
    if row==0:
        flag=0
    elif row==k-1:
        flag=1
    if flag==0:
        row+=1
    else:
        row-=1
for i in range(k):
    print("".join(enc[i]))
ct=[]
for i in range(k):
```

```
    for j in range(len(s)):
        if enc[i][j]!=' ':
            ct.append(enc[i][j])
cipher="".join(ct)
print("Cipher Text: ",cipher)
```

Pract-2 & 4 RSA algorithm & Digital Signature

```
import rsa
public_key, private_key = rsa.newkeys(1024)
print(public_key)
print('\n',private_key)
with open ("public.pem", "wb") as f:
    f.write(public_key.save_pkcs1("PEM"))
with open ("private.pem", "wb") as f:
    f.write(private_key.save_pkcs1("PEM"))
with open ("public.pem", "rb") as f:
    public_key = rsa.PublicKey.load_pkcs1(f.read())
with open ("private.pem", "rb") as f:
    private_key = rsa.PrivateKey.load_pkcs1(f.read())
message = input("\nEnter the secret message you want to share = ")
print('Select operation')
print('1: Confidentiality - Encryption ')
print('2: Confidentiality - Decryption')
print('3: Digital signature')
print('4: Exit')
while True:
    ch=int(input("Enter choice for operation = "))
    if ch==1:
        encrypted_message = rsa.encrypt(message.encode(), public_key)
```

```
        print("Encrypted message is = ", encrypted_message)
    elif ch == 2:
        encrypted_message = rsa.encrypt(message.encode(), public_key)
        decrypted_message = rsa.decrypt(encrypted_message, private_key)
        print("Decrypted message is = ", decrypted_message.decode())
    elif ch == 3:
        digi_sign = rsa.sign(message.encode(), private_key, "SHA-256")
        print("Digital signature is = ", digi_sign)
        option = input("Do you want to verify it (y/n): ")
        if option == "y":
            print(rsa.verify(message.encode(), digi_sign, public_key))
    else:
        break
```

---

pract-3 MAC

```
import hmac
import hashlib
def generate_mac(message, key):
    if isinstance(message, str):
        message = message.encode('utf-8')
    if isinstance(key, str):
        key = key.encode('utf-8')
    h = hmac.new(key, message, hashlib.sha256)
    return h.hexdigest()
def verify_mac(message, key, mac):
    generated_mac = generate_mac(message, key)
    return generated_mac == mac

message = input("Enter the message whose HMAC is to be generated ")
key = input("enter random secret key to create HMAC ")
```

```
mac = generate_mac(message, key)
print(" Generated MAC:", mac)
option=input("Do you want to verify it (y/n) ")
if option=='y':
    is_valid = verify_mac(message, key, mac)
    print("MAC is valid:", is_valid)
```

Pract-5 Diffie-Hellman Key

```
from random import randint
if __name__ == '__main__':
    p = int(input("Enter the value of prime number p: "))
    g = int(input("Enter the value of Generator g: "))
    print('Agreed Prime is: %d'%(p))
    print('And agreed generator g is: %d'%(g))
    a = int(input("Enter Alice's secret key: "))
    print('Secret number for Alice is :%d'%(g))
    PUa = int(pow(g,a,p))
    b = int(input("Enter Bob's secret key: "))
    print('Secret number for Bob is :%d'%(g))
    PUb = int(pow(p,g,b))
    ka = int(pow(PUb,a,p))
    kb = int(pow(PUa,b,p))
    print('Secret key for Alice is: %d'%(ka))
    print('Secret key for Bob is: %d'%(kb))
```